

Hybrid RBAC and ABAC Sample Dataset for a Job Offer Application

Executive Summary

The attached architectural reference argues for a hybrid authorization model because job-offer decisions depend on more than static roles: company or division scope, salary thresholds, above-band status, and contextual checks such as remote-candidate verification all affect whether a user may act. The companion admin-page mockup also expects role assignments, scope/access assignments, approval limits, effective-permission troubleshooting, and approval-routing eligibility. That combination strongly supports a seed dataset that mixes role grants with attribute-driven conditions rather than pure RBAC alone.

turn0file0turn0file1

The script below therefore seeds a realistic mid-size-company sample for the table set you named:

`JobOfferUsers`, `UserAttributes`, `UserDivisionAccess`, `Permissions`, `RolePermissions`, `AuthorizationPolicies`, and `ApprovalLevels`. It covers 20 users, 6 roles, 31 permissions, 15 ABAC policies, explicit deny rules using a deny-overrides convention, cross-division approval examples, manager chains, approval limits, and a multi-step workflow that becomes stricter for higher-value or remote offers. The design also stays PostgreSQL-friendly by using explicit column lists and by resolving dependent inserts from stable business keys such as `EmployeeNo` and `PermissionCode`, which is aligned with PostgreSQL's `INSERT ... VALUES` and `INSERT ... SELECT` syntax model. ¹

One important limitation is structural rather than logical: the visible reference material shows `UserAttributes` as a key-value store and `RolePermissions` as a role-to-permission bridge, but it does not expose a dedicated user-role table or a typed `manager_id` foreign-key column among the listed seed tables. Because of that, the sample below stores each user's effective role in `UserAttributes.primary_role` and stores `manager_id` as the referenced `JobOfferUsers.Id` rendered into the attribute store. That keeps the dataset coherent and self-contained, but the `manager_id` relationship is semantically consistent rather than declaratively enforced unless your actual schema adds a typed FK column for it. PostgreSQL foreign keys normally reference primary or unique keys in typed columns, not values hidden inside a generic attribute row. turn0file0 ²

Schema Mapping and Assumptions

I treated the accessible reference text as authoritative for the visible pieces of the design: `JobOfferUsers` holds identity-linked users, `UserAttributes` is the ABAC key-value store, `RolePermissions` bridges roles to permission rows, `AuthorizationPolicies` stores rule expressions, and `ApprovalLevels` stores ordered approval routing. The mockup's emphasis on access scopes, approval limits, generated claims, and effective permissions influenced the specific attributes chosen for the sample users.

turn0file0turn0file1

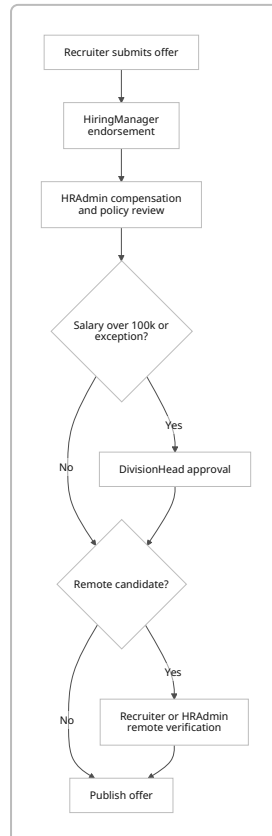
Because the ZIP DDL itself was not visible in this session, I kept assumptions deliberately narrow and documented them again inside the SQL comments. The most important ones are summarized here.

Area	Assumption used in the seed script	Why this is reasonable
<code>JobOfferUsers.Id</code> , <code>Permissions.Id</code> , <code>AuthorizationPolicies.Id</code> , <code>ApprovalLevels.Id</code>	Identity or generated PKs	The reference material shows identity-style PKs for these tables. filecite turn0file0
<code>RolePermissions.RoleId</code>	Stores stable role names such as <code>HRAdmin</code> or <code>DivisionHead</code>	The visible design keeps roles in the external identity subsystem and shows <code>RolePermissions(RoleId, PermissionId)</code> without a separate role seed table in the listed set. filecite turn0file0
User-to-role assignment	Stored as <code>UserAttributes.primary_role</code>	No dedicated user-role bridge was visible among the named tables; this avoids inventing an extra table. filecite turn0file0
<code>UserDivisionAccess</code>	Assumed to contain at least <code>(JobOfferUserId, DivisionId)</code>	This mirrors the reference's access-bridge pattern for user scope tables. filecite turn0file0
Division master data	External to the listed seed set; constant IDs are used	The visible table list does not include a <code>Divisions</code> master table, so the sample uses fixed IDs and names in comments and attributes.
Deny rules	Encoded by naming convention in <code>AuthorizationPolicies.PolicyName</code> with <code>DENY:</code> prefix	The visible policy table shape has no explicit <code>Effect</code> or <code>Priority</code> column, so deny-overrides must be represented by convention. filecite turn0file0

The result is deliberately production-like without pretending to know columns that were not visible. It uses six roles requested in your prompt—`Applicant`, `Recruiter`, `HiringManager`, `HRAdmin`, `DivisionHead`, and `SystemAdmin`—while preserving the reference design's broader ABAC ideas of salary limits, division scope, approval routing, and exception handling. [filecite turn0file0](#)
[filecite turn0file1](#)

Approval Flow Illustration

The high-value path below is the sample workflow encoded by the seed data and validation queries: the recruiter submits, the hiring manager endorses, HR performs compensation and policy checks, Division Head approval activates when the offer is high-value or exceptional, and remote candidates must complete verification before publication. That mirrors the approval-routing and approval-limit concepts in the reference material while adapting them to the six-role set requested in your prompt.



SQL Script

```
-- Hybrid RBAC + ABAC sample seed data for a Job Offer application
-- PostgreSQL-compatible INSERT script
-- Assumptions:
-- 1) JobOfferUsers.Id, Permissions.Id, AuthorizationPolicies.Id,
ApprovalLevels.Id use identity/generated keys,
-- consistent with the attached reference design.
-- 2) RolePermissions.RoleId stores stable role names. If your implementation
stores AspNetRoles.Id GUIDs instead,
-- replace the role-name literals in the VALUES block with your actual role
ids.
-- 3) UserDivisionAccess is assumed to expose at least (JobOfferUserId,
```

```

DivisionId). If your actual table has a
-- surrogate key or audit columns, append defaults accordingly.
-- 4) Division master data is external to the provided table set. This script
uses constant division ids:
--      10 = HR, 20 = Engineering, 30 = Sales, 40 = Finance, 50 = Operations,
90 = Corporate
-- 5) Because AuthorizationPolicies has no explicit Effect/Priority columns in
the reference design,
-- deny rules are encoded by convention: PolicyName prefixed with 'DENY:'
represents an explicit deny.
-- Effective evaluation is deny-overrides.
-- 6) Approval thresholds are expressed primarily in ABAC policies and user
approval_limit attributes because the
-- visible ApprovalLevels design focuses on routing sequence rather than
threshold columns.

```

```
BEGIN;
```

```

-----
-- JobOfferUsers
-- 20 users across multiple divisions and roles for a mid-size company.
-- Two applicant accounts are inactive to support edge-case validation.
-----
INSERT INTO JobOfferUsers
(IdentityUserId, EmployeeNo, Email, FullName, IsActive, CreatedDate)
VALUES
('11111111-1111-1111-1111-111111111001', 'EMP001',
'ava.patel@midmarket.example', 'Ava Patel', TRUE, '2026-01-03 09:00:00'),
('11111111-1111-1111-1111-111111111002', 'EMP002',
'olivia.chen@midmarket.example', 'Olivia Chen', TRUE, '2026-01-04 09:00:00'),
('11111111-1111-1111-1111-111111111003', 'EMP003',
'ethan.brooks@midmarket.example', 'Ethan Brooks', TRUE, '2026-01-05 09:00:00'),
('11111111-1111-1111-1111-111111111004', 'EMP004',
'sophia.kim@midmarket.example', 'Sophia Kim', TRUE, '2026-01-06 09:00:00'),
('11111111-1111-1111-1111-111111111005', 'EMP005',
'liam.turner@midmarket.example', 'Liam Turner', TRUE, '2026-01-07 09:00:00'),
('11111111-1111-1111-1111-111111111006', 'EMP006',
'mia.gonzales@midmarket.example', 'Mia Gonzales', TRUE, '2026-01-08 09:00:00'),
('11111111-1111-1111-1111-111111111007', 'EMP007',
'chloe.rivera@midmarket.example', 'Chloe Rivera', TRUE, '2026-01-09 09:00:00'),
('11111111-1111-1111-1111-111111111008', 'EMP008',
'daniel.ross@midmarket.example', 'Daniel Ross', TRUE, '2026-01-10 09:00:00'),
('11111111-1111-1111-1111-111111111009', 'EMP009',
'hannah.white@midmarket.example', 'Hannah White', TRUE, '2026-01-11 09:00:00'),
('11111111-1111-1111-1111-111111111010', 'EMP010',
'jacob.miller@midmarket.example', 'Jacob Miller', TRUE, '2026-01-12 09:00:00'),
('11111111-1111-1111-1111-111111111011', 'EMP011',
'zoe.bennett@midmarket.example', 'Zoe Bennett', TRUE, '2026-01-13 09:00:00'),

```

```

        ('11111111-1111-1111-1111-111111111012', 'EMP012',
'amelia.torres@midmarket.example', 'Amelia Torres', TRUE, '2026-01-14
09:00:00'),
        ('11111111-1111-1111-1111-111111111013', 'EMP013',
'victor.hall@midmarket.example', 'Victor Hall', TRUE, '2026-01-15 09:00:00'),
        ('11111111-1111-1111-1111-111111111014', 'EMP014',
'isla.reed@midmarket.example', 'Isla Reed', TRUE, '2026-01-16 09:00:00'),
        ('11111111-1111-1111-1111-111111111015', 'EMP015',
'caleb.stone@midmarket.example', 'Caleb Stone', TRUE, '2026-01-17 09:00:00'),
        ('11111111-1111-1111-1111-111111111016', 'EMP016',
'emily.davis@midmarket.example', 'Emily Davis', TRUE, '2026-01-18 09:00:00'),
        ('11111111-1111-1111-1111-111111111017', 'EMP017',
'mason.clark@midmarket.example', 'Mason Clark', FALSE, '2026-01-19 09:00:00'),
        ('11111111-1111-1111-1111-111111111018', 'EMP018',
'lily.adams@midmarket.example', 'Lily Adams', TRUE, '2026-01-20 09:00:00'),
        ('11111111-1111-1111-1111-111111111019', 'EMP019',
'owen.scott@midmarket.example', 'Owen Scott', FALSE, '2026-01-21 09:00:00'),
        ('11111111-1111-1111-1111-111111111020', 'EMP020',
'harper.james@midmarket.example', 'Harper James', TRUE, '2026-01-22 09:00:00')
;

```

```

-----
-- Permissions
-- 31 permissions spanning offer lifecycle, salary handling, assignment,
-- approvals, compliance checks, admin functions, and troubleshooting.
-----

```

```

INSERT INTO Permissions
    (PermissionCode, PermissionName)
VALUES
    ('create_offer', 'Create Offer'),
    ('view_offer', 'View Offer'),
    ('edit_offer', 'Edit Offer'),
    ('delete_offer', 'Delete Offer'),
    ('submit_offer', 'Submit Offer'),
    ('withdraw_offer', 'Withdraw Offer'),
    ('approve_offer', 'Approve Offer'),
    ('reject_offer', 'Reject Offer'),
    ('return_offer', 'Return Offer'),
    ('publish_offer', 'Publish Offer'),
    ('revoke_offer', 'Revoke Offer'),
    ('view_salary', 'View Salary'),
    ('edit_salary', 'Edit Salary'),
    ('view_candidate', 'View Candidate'),
    ('edit_candidate', 'Edit Candidate'),
    ('assign_recruiter', 'Assign Recruiter'),
    ('assign_hiring_manager', 'Assign Hiring Manager'),
    ('request_exception', 'Request Exception'),
    ('escalate', 'Escalate'),

```

```

('verify_remote_work', 'Verify Remote Work'),
('verify_background', 'Verify Background'),
('view_audit_logs', 'View Audit Logs'),
('manage_policies', 'Manage Policies'),
('manage_roles', 'Manage Roles'),
('manage_users', 'Manage Users'),
('simulate_authorization', 'Simulate Authorization'),
('view_reports', 'View Reports'),
('export_offer', 'Export Offer'),
('comment_offer', 'Comment Offer'),
('route_offer', 'Route Offer'),
('override_policy', 'Override Policy')
;

-----
-- RolePermissions
-- Role names are treated as stable identifiers because no explicit Roles
-- seed table was included in the listed table set.
-----

INSERT INTO RolePermissions
  (RoleId, PermissionId)
SELECT
  v.RoleId,
  p.Id
FROM (
VALUES
  ('Applicant', 'comment_offer'),
  ('Applicant', 'view_candidate'),
  ('Applicant', 'view_offer'),
  ('Applicant', 'withdraw_offer'),

  ('Recruiter', 'comment_offer'),
  ('Recruiter', 'create_offer'),
  ('Recruiter', 'edit_candidate'),
  ('Recruiter', 'edit_offer'),
  ('Recruiter', 'escalate'),
  ('Recruiter', 'request_exception'),
  ('Recruiter', 'return_offer'),
  ('Recruiter', 'route_offer'),
  ('Recruiter', 'submit_offer'),
  ('Recruiter', 'verify_background'),
  ('Recruiter', 'verify_remote_work'),
  ('Recruiter', 'view_candidate'),
  ('Recruiter', 'view_offer'),
  ('Recruiter', 'view_reports'),
  ('Recruiter', 'view_salary'),
  ('Recruiter', 'withdraw_offer'),
  ('Recruiter', 'assign_hiring_manager'),

```

```

('HiringManager', 'approve_offer'),
('HiringManager', 'comment_offer'),
('HiringManager', 'edit_offer'),
('HiringManager', 'reject_offer'),
('HiringManager', 'request_exception'),
('HiringManager', 'return_offer'),
('HiringManager', 'route_offer'),
('HiringManager', 'view_candidate'),
('HiringManager', 'view_offer'),
('HiringManager', 'view_reports'),
('HiringManager', 'view_salary'),

('HRAdmin', 'approve_offer'),
('HRAdmin', 'assign_hiring_manager'),
('HRAdmin', 'assign_recruiter'),
('HRAdmin', 'comment_offer'),
('HRAdmin', 'create_offer'),
('HRAdmin', 'delete_offer'),
('HRAdmin', 'edit_candidate'),
('HRAdmin', 'edit_offer'),
('HRAdmin', 'edit_salary'),
('HRAdmin', 'escalate'),
('HRAdmin', 'export_offer'),
('HRAdmin', 'publish_offer'),
('HRAdmin', 'reject_offer'),
('HRAdmin', 'request_exception'),
('HRAdmin', 'return_offer'),
('HRAdmin', 'revoke_offer'),
('HRAdmin', 'route_offer'),
('HRAdmin', 'simulate_authorization'),
('HRAdmin', 'submit_offer'),
('HRAdmin', 'verify_background'),
('HRAdmin', 'verify_remote_work'),
('HRAdmin', 'view_audit_logs'),
('HRAdmin', 'view_candidate'),
('HRAdmin', 'view_offer'),
('HRAdmin', 'view_reports'),
('HRAdmin', 'view_salary'),

('DivisionHead', 'approve_offer'),
('DivisionHead', 'comment_offer'),
('DivisionHead', 'escalate'),
('DivisionHead', 'export_offer'),
('DivisionHead', 'publish_offer'),
('DivisionHead', 'reject_offer'),
('DivisionHead', 'request_exception'),
('DivisionHead', 'return_offer'),

```

```

('DivisionHead', 'route_offer'),
('DivisionHead', 'view_candidate'),
('DivisionHead', 'view_offer'),
('DivisionHead', 'view_reports'),
('DivisionHead', 'view_salary'),

('SystemAdmin', 'export_offer'),
('SystemAdmin', 'manage_policies'),
('SystemAdmin', 'manage_roles'),
('SystemAdmin', 'manage_users'),
('SystemAdmin', 'override_policy'),
('SystemAdmin', 'simulate_authorization'),
('SystemAdmin', 'view_audit_logs'),
('SystemAdmin', 'view_offer'),
('SystemAdmin', 'view_reports')
) AS v(RoleId, PermissionCode)
JOIN Permissions p
  ON p.PermissionCode = v.PermissionCode
;

-----
-- UserDivisionAccess
-- Division access drives ABAC scope checks.
-- Engineering = 20, Sales = 30, Finance = 40, Operations = 50.
-- Amelia Torres (Finance DivisionHead) intentionally has cross-division
-- access to Engineering to model compensation-committee approval.
-----

INSERT INTO UserDivisionAccess
  (JobOfferUserId, DivisionId)
SELECT
  ju.Id,
  v.DivisionId
FROM (
VALUES
  ('EMP001', 10),
  ('EMP001', 20),
  ('EMP001', 30),
  ('EMP001', 40),
  ('EMP001', 50),
  ('EMP001', 90),

  ('EMP002', 10),
  ('EMP002', 20),
  ('EMP002', 30),
  ('EMP002', 40),
  ('EMP002', 50),

  ('EMP003', 10),

```



```

('EMP003', 20),
('EMP003', 30),
('EMP003', 40),
('EMP003', 50),

('EMP004', 20),
('EMP005', 20),
('EMP006', 20),

('EMP007', 20),
('EMP007', 40),

('EMP008', 30),
('EMP008', 50),

('EMP009', 30),
('EMP010', 30),
('EMP011', 30),

('EMP012', 40),
('EMP012', 20),

('EMP013', 40),

('EMP014', 50),
('EMP015', 50),

('EMP016', 20),
('EMP017', 20),
('EMP018', 30),
('EMP019', 40),
('EMP020', 50)
) AS v(EmployeeNo, DivisionId)
JOIN JobOfferUsers ju
    ON ju.EmployeeNo = v.EmployeeNo
;

-- -----
-- UserAttributes
-- ABAC attributes stored as key-value rows. manager_id is resolved to the
-- referenced JobOfferUsers.Id and written as text into the attribute store.
-- This preserves coherence even though the key-value pattern cannot enforce
-- a native FK on manager_id without schema extension.
-- -----
INSERT INTO UserAttributes
    (JobOfferUserId, AttributeName, AttributeValue)
SELECT
    ju.Id,

```

```

v.AttributeName,
CASE
    WHEN v.AttributeName = 'manager_id' AND v.AttributeValue IS NOT NULL
        THEN (
            SELECT mgr.Id::text
            FROM JobOfferUsers mgr
            WHERE mgr.EmployeeNo = v.AttributeValue
        )
    ELSE v.AttributeValue
END AS AttributeValue
FROM (
VALUES
    ('EMP001', 'primary_role', 'SystemAdmin'),
    ('EMP001', 'division_id', '90'),
    ('EMP001', 'division_name', 'Corporate'),
    ('EMP001', 'employment_level', 'L6'),
    ('EMP001', 'clearance_level', '5'),
    ('EMP001', 'location', 'US-NY'),
    ('EMP001', 'tenure_years', '9'),
    ('EMP001', 'certifications', 'CISSP|ITIL'),
    ('EMP001', 'manager_id', NULL),
    ('EMP001', 'approval_limit', '0'),
    ('EMP001', 'country_scope', 'ALL'),
    ('EMP001', 'remote_verifier', 'false'),

    ('EMP002', 'primary_role', 'HRAdmin'),
    ('EMP002', 'division_id', '10'),
    ('EMP002', 'division_name', 'HR'),
    ('EMP002', 'employment_level', 'L5'),
    ('EMP002', 'clearance_level', '4'),
    ('EMP002', 'location', 'US-IL'),
    ('EMP002', 'tenure_years', '8'),
    ('EMP002', 'certifications', 'SHRM-SCP|CCP'),
    ('EMP002', 'manager_id', NULL),
    ('EMP002', 'approval_limit', '150000'),
    ('EMP002', 'country_scope', 'US|PH|SG'),
    ('EMP002', 'remote_verifier', 'true'),

    ('EMP003', 'primary_role', 'HRAdmin'),
    ('EMP003', 'division_id', '10'),
    ('EMP003', 'division_name', 'HR'),
    ('EMP003', 'employment_level', 'L4'),
    ('EMP003', 'clearance_level', '4'),
    ('EMP003', 'location', 'PH-MNL'),
    ('EMP003', 'tenure_years', '6'),
    ('EMP003', 'certifications', 'SHRM-CP|Compensation Analyst'),
    ('EMP003', 'manager_id', 'EMP002'),
    ('EMP003', 'approval_limit', '100000'),

```

```

('EMP003', 'country_scope', 'PH|SG'),
('EMP003', 'remote_verifier', 'true'),

('EMP004', 'primary_role', 'DivisionHead'),
('EMP004', 'division_id', '20'),
('EMP004', 'division_name', 'Engineering'),
('EMP004', 'employment_level', 'L5'),
('EMP004', 'clearance_level', '4'),
('EMP004', 'location', 'US-CA'),
('EMP004', 'tenure_years', '7'),
('EMP004', 'certifications', 'PMP|SAFe'),
('EMP004', 'manager_id', 'EMP002'),
('EMP004', 'approval_limit', '250000'),
('EMP004', 'country_scope', 'US|SG'),
('EMP004', 'remote_verifier', 'false'),

('EMP005', 'primary_role', 'HiringManager'),
('EMP005', 'division_id', '20'),
('EMP005', 'division_name', 'Engineering'),
('EMP005', 'employment_level', 'L4'),
('EMP005', 'clearance_level', '3'),
('EMP005', 'location', 'US-TX'),
('EMP005', 'tenure_years', '5'),
('EMP005', 'certifications', 'AWS-SAA|CSM'),
('EMP005', 'manager_id', 'EMP004'),
('EMP005', 'approval_limit', '120000'),
('EMP005', 'country_scope', 'US'),
('EMP005', 'remote_verifier', 'false'),

('EMP006', 'primary_role', 'HiringManager'),
('EMP006', 'division_id', '20'),
('EMP006', 'division_name', 'Engineering'),
('EMP006', 'employment_level', 'L4'),
('EMP006', 'clearance_level', '3'),
('EMP006', 'location', 'SG-SIN'),
('EMP006', 'tenure_years', '4'),
('EMP006', 'certifications', 'AWS-DVA|ScrumMaster'),
('EMP006', 'manager_id', 'EMP004'),
('EMP006', 'approval_limit', '110000'),
('EMP006', 'country_scope', 'SG|PH'),
('EMP006', 'remote_verifier', 'false'),

('EMP007', 'primary_role', 'Recruiter'),
('EMP007', 'division_id', '10'),
('EMP007', 'division_name', 'HR'),
('EMP007', 'employment_level', 'L3'),
('EMP007', 'clearance_level', '3'),
('EMP007', 'location', 'US-IL'),

```

```

('EMP007', 'tenure_years', '4'),
('EMP007', 'certifications', 'CIR|LinkedInRecruiter'),
('EMP007', 'manager_id', 'EMP002'),
('EMP007', 'approval_limit', '0'),
('EMP007', 'country_scope', 'US|PH'),
('EMP007', 'remote_verifier', 'true'),

('EMP008', 'primary_role', 'Recruiter'),
('EMP008', 'division_id', '10'),
('EMP008', 'division_name', 'HR'),
('EMP008', 'employment_level', 'L3'),
('EMP008', 'clearance_level', '3'),
('EMP008', 'location', 'PH-MNL'),
('EMP008', 'tenure_years', '3'),
('EMP008', 'certifications', 'CIR|BooleanSearch'),
('EMP008', 'manager_id', 'EMP003'),
('EMP008', 'approval_limit', '0'),
('EMP008', 'country_scope', 'PH|SG'),
('EMP008', 'remote_verifier', 'true'),

('EMP009', 'primary_role', 'DivisionHead'),
('EMP009', 'division_id', '30'),
('EMP009', 'division_name', 'Sales'),
('EMP009', 'employment_level', 'L5'),
('EMP009', 'clearance_level', '4'),
('EMP009', 'location', 'US-FL'),
('EMP009', 'tenure_years', '9'),
('EMP009', 'certifications', 'CPSP|MEDDICC'),
('EMP009', 'manager_id', 'EMP002'),
('EMP009', 'approval_limit', '220000'),
('EMP009', 'country_scope', 'US|UK'),
('EMP009', 'remote_verifier', 'false'),

('EMP010', 'primary_role', 'HiringManager'),
('EMP010', 'division_id', '30'),
('EMP010', 'division_name', 'Sales'),
('EMP010', 'employment_level', 'L4'),
('EMP010', 'clearance_level', '3'),
('EMP010', 'location', 'US-NY'),
('EMP010', 'tenure_years', '5'),
('EMP010', 'certifications', 'MEDDICC|SPIN'),
('EMP010', 'manager_id', 'EMP009'),
('EMP010', 'approval_limit', '90000'),
('EMP010', 'country_scope', 'US'),
('EMP010', 'remote_verifier', 'false'),

('EMP011', 'primary_role', 'HiringManager'),
('EMP011', 'division_id', '30'),

```

```

('EMP011', 'division_name', 'Sales'),
('EMP011', 'employment_level', 'L4'),
('EMP011', 'clearance_level', '3'),
('EMP011', 'location', 'UK-LON'),
('EMP011', 'tenure_years', '6'),
('EMP011', 'certifications', 'MEDDPICC|NegotiationPro'),
('EMP011', 'manager_id', 'EMP009'),
('EMP011', 'approval_limit', '95000'),
('EMP011', 'country_scope', 'UK|US'),
('EMP011', 'remote_verifier', 'false'),

('EMP012', 'primary_role', 'DivisionHead'),
('EMP012', 'division_id', '40'),
('EMP012', 'division_name', 'Finance'),
('EMP012', 'employment_level', 'L5'),
('EMP012', 'clearance_level', '4'),
('EMP012', 'location', 'PH-MNL'),
('EMP012', 'tenure_years', '10'),
('EMP012', 'certifications', 'CPA|CMA'),
('EMP012', 'manager_id', 'EMP002'),
('EMP012', 'approval_limit', '230000'),
('EMP012', 'country_scope', 'PH|US'),
('EMP012', 'remote_verifier', 'false'),

('EMP013', 'primary_role', 'HiringManager'),
('EMP013', 'division_id', '40'),
('EMP013', 'division_name', 'Finance'),
('EMP013', 'employment_level', 'L4'),
('EMP013', 'clearance_level', '3'),
('EMP013', 'location', 'PH-CEB'),
('EMP013', 'tenure_years', '7'),
('EMP013', 'certifications', 'CPA|FMVA'),
('EMP013', 'manager_id', 'EMP012'),
('EMP013', 'approval_limit', '85000'),
('EMP013', 'country_scope', 'PH'),
('EMP013', 'remote_verifier', 'false'),

('EMP014', 'primary_role', 'DivisionHead'),
('EMP014', 'division_id', '50'),
('EMP014', 'division_name', 'Operations'),
('EMP014', 'employment_level', 'L5'),
('EMP014', 'clearance_level', '4'),
('EMP014', 'location', 'SG-SIN'),
('EMP014', 'tenure_years', '8'),
('EMP014', 'certifications', 'LeanSixSigmaBB|PMP'),
('EMP014', 'manager_id', 'EMP002'),
('EMP014', 'approval_limit', '200000'),
('EMP014', 'country_scope', 'SG|PH'),

```

```

('EMP014', 'remote_verifier', 'false'),

('EMP015', 'primary_role', 'HiringManager'),
('EMP015', 'division_id', '50'),
('EMP015', 'division_name', 'Operations'),
('EMP015', 'employment_level', 'L4'),
('EMP015', 'clearance_level', '3'),
('EMP015', 'location', 'SG-SIN'),
('EMP015', 'tenure_years', '5'),
('EMP015', 'certifications', 'LeanSixSigmaGB|APICS'),
('EMP015', 'manager_id', 'EMP014'),
('EMP015', 'approval_limit', '80000'),
('EMP015', 'country_scope', 'SG'),
('EMP015', 'remote_verifier', 'false'),

('EMP016', 'primary_role', 'Applicant'),
('EMP016', 'division_id', '20'),
('EMP016', 'division_name', 'Engineering'),
('EMP016', 'employment_level', 'L1'),
('EMP016', 'clearance_level', '1'),
('EMP016', 'location', 'US-Remote'),
('EMP016', 'tenure_years', '0'),
('EMP016', 'certifications', 'AWS-CCP'),
('EMP016', 'manager_id', NULL),
('EMP016', 'approval_limit', '0'),
('EMP016', 'country_scope', 'US'),
('EMP016', 'remote_verifier', 'false'),

('EMP017', 'primary_role', 'Applicant'),
('EMP017', 'division_id', '20'),
('EMP017', 'division_name', 'Engineering'),
('EMP017', 'employment_level', 'L1'),
('EMP017', 'clearance_level', '1'),
('EMP017', 'location', 'PH-Remote'),
('EMP017', 'tenure_years', '0'),
('EMP017', 'certifications', 'None'),
('EMP017', 'manager_id', NULL),
('EMP017', 'approval_limit', '0'),
('EMP017', 'country_scope', 'PH'),
('EMP017', 'remote_verifier', 'false'),

('EMP018', 'primary_role', 'Applicant'),
('EMP018', 'division_id', '30'),
('EMP018', 'division_name', 'Sales'),
('EMP018', 'employment_level', 'L1'),
('EMP018', 'clearance_level', '1'),
('EMP018', 'location', 'UK-Remote'),
('EMP018', 'tenure_years', '0'),

```

```

('EMP018', 'certifications', 'None'),
('EMP018', 'manager_id', NULL),
('EMP018', 'approval_limit', '0'),
('EMP018', 'country_scope', 'UK'),
('EMP018', 'remote_verifier', 'false'),

('EMP019', 'primary_role', 'Applicant'),
('EMP019', 'division_id', '40'),
('EMP019', 'division_name', 'Finance'),
('EMP019', 'employment_level', 'L1'),
('EMP019', 'clearance_level', '1'),
('EMP019', 'location', 'PH-MNL'),
('EMP019', 'tenure_years', '0'),
('EMP019', 'certifications', 'None'),
('EMP019', 'manager_id', NULL),
('EMP019', 'approval_limit', '0'),
('EMP019', 'country_scope', 'PH'),
('EMP019', 'remote_verifier', 'false'),

('EMP020', 'primary_role', 'Applicant'),
('EMP020', 'division_id', '50'),
('EMP020', 'division_name', 'Operations'),
('EMP020', 'employment_level', 'L1'),
('EMP020', 'clearance_level', '1'),
('EMP020', 'location', 'SG-Remote'),
('EMP020', 'tenure_years', '0'),
('EMP020', 'certifications', 'None'),
('EMP020', 'manager_id', NULL),
('EMP020', 'approval_limit', '0'),
('EMP020', 'country_scope', 'SG'),
('EMP020', 'remote_verifier', 'false')
) AS v(EmployeeNo, AttributeName, AttributeValue)
JOIN JobOfferUsers ju
  ON ju.EmployeeNo = v.EmployeeNo
;

-- -----
-- AuthorizationPolicies
-- Policy expressions are stored as text and are intended to be interpreted
-- by the application's ABAC engine. DENY: rows are explicit denies and
-- should be evaluated before ALLOW: rows.
-- -----

INSERT INTO AuthorizationPolicies
  (PolicyName, PermissionCode, Expression, IsActive)
VALUES
  ('ALLOW: Recruiter create_offer in assigned divisions',
   'create_offer',
   'ALLOW IF User.primary_role = ''Recruiter'' AND User.is_active = true AND

```

```

Resource.division_id IN User.division_access',
    TRUE),

('ALLOW: HiringManager approve within division and approval_limit',
 'approve_offer',
 'ALLOW IF User.primary_role = ''HiringManager'' AND User.is_active = true
AND Resource.division_id IN User.division_access AND Resource.salary <=
User.approval_limit AND Resource.candidate_country IN User.country_scope',
    TRUE),

('ALLOW: HRAdmin approve cross-division within approval_limit',
 'approve_offer',
 'ALLOW IF User.primary_role = ''HRAdmin'' AND User.is_active = true AND
Resource.salary <= User.approval_limit AND Resource.candidate_country IN
User.country_scope',
    TRUE),

('ALLOW: DivisionHead approve high-value in covered divisions',
 'approve_offer',
 'ALLOW IF User.primary_role = ''DivisionHead'' AND User.is_active = true
AND Resource.division_id IN User.division_access AND Resource.salary <=
User.approval_limit AND User.clearance_level >= 4 AND Resource.candidate_country
IN User.country_scope',
    TRUE),

('DENY: approve_offer above personal approval_limit',
 'approve_offer',
 'DENY IF Resource.salary > User.approval_limit',
    TRUE),

('DENY: DivisionHead out-of-division approval',
 'approve_offer',
 'DENY IF User.primary_role = ''DivisionHead'' AND Resource.division_id NOT
IN User.division_access',
    TRUE),

('ALLOW: view_salary with clearance >= 3',
 'view_salary',
 'ALLOW IF User.clearance_level >= 3 AND User.is_active = true',
    TRUE),

('DENY: view_salary with clearance < 3',
 'view_salary',
 'DENY IF User.clearance_level < 3',
    TRUE),

('DENY: edit_salary for Recruiter and Applicant',
 'edit_salary',

```



```

'DENY IF User.primary_role IN ('Recruiter','Applicant')',
TRUE),

('ALLOW: edit_salary for HRAdmin',
'edit_salary',
'ALLOW IF User.primary_role = 'HRAdmin' AND User.is_active = true',
TRUE),

('ALLOW: verify_remote_work by Recruiter or HRAdmin',
'verify_remote_work',
'ALLOW IF User.primary_role IN ('Recruiter','HRAdmin') AND
User.remote_verifier = true AND User.is_active = true',
TRUE),

('DENY: publish_offer until remote candidate is verified',
'publish_offer',
'DENY IF Resource.is_remote = true AND Resource.remote_verified = false',
TRUE),

('ALLOW: publish_offer by HRAdmin after approvals complete',
'publish_offer',

'ALLOW IF User.primary_role = 'HRAdmin' AND Resource.approvals_complete = true
AND User.is_active = true',
TRUE),

('DENY: override_policy for non-SystemAdmin',
'override_policy',
'DENY IF User.primary_role <> 'SystemAdmin'',
TRUE),

('ALLOW: SystemAdmin security administration',
'manage_policies',
'ALLOW IF User.primary_role = 'SystemAdmin' AND User.is_active = true',
TRUE)
;

-- -----
-- ApprovalLevels
-- Since the visible schema does not expose threshold columns, threshold
-- logic is enforced by ABAC policies and approval_limit attributes.
-- Sequence simply defines the nominal routing order.
-- -----
INSERT INTO ApprovalLevels
(SequenceNo, ApprovalRole, IsAboveBandOnly)
VALUES
(1, 'HiringManager', FALSE),
(2, 'HRAdmin', FALSE),

```

```

        (3, 'DivisionHead', FALSE)
    ;

COMMIT;

-----
-- Seed-specific counts summary
-- Counts only the rows inserted by this seed, assuming no duplicate reruns.
-----

WITH seeded_users AS (
    SELECT Id
    FROM JobOfferUsers
    WHERE EmployeeNo IN (
        'EMP001', 'EMP002', 'EMP003', 'EMP004', 'EMP005',
        'EMP006', 'EMP007', 'EMP008', 'EMP009', 'EMP010',
        'EMP011', 'EMP012', 'EMP013', 'EMP014', 'EMP015',
        'EMP016', 'EMP017', 'EMP018', 'EMP019', 'EMP020'
    )
),
seeded_permissions AS (
    SELECT Id
    FROM Permissions
    WHERE PermissionCode IN (
        'create_offer', 'view_offer', 'edit_offer', 'delete_offer',
        'submit_offer', 'withdraw_offer',
        'approve_offer', 'reject_offer', 'return_offer', 'publish_offer',
        'revoke_offer',
        'view_salary', 'edit_salary', 'view_candidate', 'edit_candidate',
        'assign_recruiter',
        'assign_hiring_manager', 'request_exception', 'escalate',
        'verify_remote_work',
        'verify_background', 'view_audit_logs', 'manage_policies',
        'manage_roles', 'manage_users',
        'simulate_authorization', 'view_reports', 'export_offer',
        'comment_offer', 'route_offer',
        'override_policy'
    )
)
SELECT 'JobOfferUsers' AS table_name, COUNT(*) AS inserted_rows
FROM JobOfferUsers
WHERE EmployeeNo IN (
    'EMP001', 'EMP002', 'EMP003', 'EMP004', 'EMP005',
    'EMP006', 'EMP007', 'EMP008', 'EMP009', 'EMP010',
    'EMP011', 'EMP012', 'EMP013', 'EMP014', 'EMP015',
    'EMP016', 'EMP017', 'EMP018', 'EMP019', 'EMP020'
)
UNION ALL
SELECT 'UserAttributes', COUNT(*)

```

```

FROM UserAttributes
WHERE JobOfferUserId IN (SELECT Id FROM seeded_users)
UNION ALL
SELECT 'UserDivisionAccess', COUNT(*)
FROM UserDivisionAccess
WHERE JobOfferUserId IN (SELECT Id FROM seeded_users)
UNION ALL
SELECT 'Permissions', COUNT(*)
FROM Permissions
WHERE PermissionCode IN (
    'create_offer', 'view_offer', 'edit_offer', 'delete_offer', 'submit_offer',
    'withdraw_offer',
    'approve_offer', 'reject_offer', 'return_offer', 'publish_offer',
    'revoke_offer',
    'view_salary', 'edit_salary', 'view_candidate', 'edit_candidate',
    'assign_recruiter',
    'assign_hiring_manager', 'request_exception', 'escalate',
    'verify_remote_work',
    'verify_background', 'view_audit_logs', 'manage_policies', 'manage_roles',
    'manage_users',
    'simulate_authorization', 'view_reports', 'export_offer', 'comment_offer',
    'route_offer',
    'override_policy'
)
UNION ALL
SELECT 'RolePermissions', COUNT(*)
FROM RolePermissions
WHERE RoleId IN ('Applicant', 'Recruiter', 'HiringManager', 'HRAdmin',
'DivisionHead', 'SystemAdmin')
    AND PermissionId IN (SELECT Id FROM seeded_permissions)
UNION ALL
SELECT 'AuthorizationPolicies', COUNT(*)
FROM AuthorizationPolicies
WHERE PolicyName IN (
    'ALLOW: Recruiter create_offer in assigned divisions',
    'ALLOW: HiringManager approve within division and approval_limit',
    'ALLOW: HRAdmin approve cross-division within approval_limit',
    'ALLOW: DivisionHead approve high-value in covered divisions',
    'DENY: approve_offer above personal approval_limit',
    'DENY: DivisionHead out-of-division approval',
    'ALLOW: view_salary with clearance >= 3',
    'DENY: view_salary with clearance < 3',
    'DENY: edit_salary for Recruiter and Applicant',
    'ALLOW: edit_salary for HRAdmin',
    'ALLOW: verify_remote_work by Recruiter or HRAdmin',
    'DENY: publish_offer until remote candidate is verified',
    'ALLOW: publish_offer by HRAdmin after approvals complete',
    'DENY: override_policy for non-SystemAdmin',

```

```

        'ALLOW: SystemAdmin security administration'
    )
UNION ALL
SELECT 'ApprovalLevels', COUNT(*)
FROM ApprovalLevels
WHERE (SequenceNo, ApprovalRole) IN ((1, 'HiringManager'), (2, 'HRAdmin'), (3,
'DivisionHead'))
ORDER BY table_name
;

-- =====
-- Validation query 1
-- Which active users can approve a USD 150,000 offer for an Engineering
-- candidate located in the US, after RBAC grants and ABAC deny-overrides?
-- Expected sample result: Olivia Chen, Sophia Kim, Amelia Torres.
-- =====
WITH offer_input AS (
    SELECT
        20::int AS division_id,
        150000::numeric(18,2) AS salary,
        'US'::text AS candidate_country,
        FALSE::boolean AS is_remote,
        FALSE::boolean AS remote_verified,
        FALSE::boolean AS above_band,
        TRUE::boolean AS approvals_complete
),
user_profile AS (
    SELECT
        ju.Id AS user_id,
        ju.EmployeeNo,
        ju.FullName,
        ju.IsActive,
        MAX(CASE WHEN ua.AttributeName = 'primary_role' THEN ua.AttributeValue
END) AS primary_role,
        MAX(CASE WHEN ua.AttributeName = 'clearance_level' THEN
ua.AttributeValue END)::int AS clearance_level,
        MAX(CASE WHEN ua.AttributeName = 'approval_limit' THEN
ua.AttributeValue END)::numeric(18,2) AS approval_limit,
        MAX(CASE WHEN ua.AttributeName = 'country_scope' THEN ua.AttributeValue
END) AS country_scope
    FROM JobOfferUsers ju
    JOIN UserAttributes ua
        ON ua.JobOfferUserId = ju.Id
    GROUP BY ju.Id, ju.EmployeeNo, ju.FullName, ju.IsActive
),
eligible AS (
    SELECT
        up.*,

```

```

        oi.*,
        EXISTS (
            SELECT 1
            FROM RolePermissions rp
            JOIN Permissions p
              ON p.Id = rp.PermissionId
            WHERE rp.RoleId = up.primary_role
              AND p.PermissionCode = 'approve_offer'
        ) AS has_rbac_approve,
        EXISTS (
            SELECT 1
            FROM UserDivisionAccess uda
            WHERE uda.JobOfferUserId = up.user_id
              AND uda.DivisionId = oi.division_id
        ) AS has_division_access
    FROM user_profile up
    CROSS JOIN offer_input oi
)
SELECT
    FullName,
    EmployeeNo,
    primary_role,
    approval_limit,
    CASE
        WHEN primary_role = 'HRAdmin' THEN 'Cross-division HR approval within
limit'
        WHEN primary_role = 'DivisionHead' THEN 'Division-head approval for
covered division'
        WHEN primary_role = 'HiringManager' THEN
'Hiring-manager approval within limit'
        ELSE 'Not eligible'
    END AS approval_basis
FROM eligible
WHERE IsActive = TRUE
    AND has_rbac_approve = TRUE
    AND salary <= approval_limit
    AND (
        country_scope = 'ALL'
        OR candidate_country = ANY(string_to_array(country_scope, '|'))
    )
    AND (
        primary_role = 'HRAdmin'
        OR (primary_role IN ('HiringManager', 'DivisionHead') AND
has_division_access = TRUE)
    )
    AND NOT (primary_role = 'DivisionHead' AND has_division_access = FALSE) --
explicit deny
ORDER BY

```

```

CASE primary_role
    WHEN 'DivisionHead' THEN 1
    WHEN 'HRAdmin' THEN 2
    WHEN 'HiringManager' THEN 3
    ELSE 9
END,
FullName
;

-- =====
-- Validation query 2
-- List effective permissions for one user, combining RBAC grants with
-- ABAC conditions and explicit deny rules. Change EMP007 to any employee.
-- Scenario: Chloe Rivera reviewing a USD 120,000 remote Engineering offer
-- for a US candidate whose remote verification is still pending.
-- =====
WITH scenario AS (
    SELECT
        'EMP007'::text AS employee_no,
        20::int AS division_id,
        120000::numeric(18,2) AS salary,
        'US'::text AS candidate_country,
        TRUE::boolean AS is_remote,
        FALSE::boolean AS remote_verified,
        FALSE::boolean AS approvals_complete
),
user_profile AS (
    SELECT
        ju.Id AS user_id,
        ju.EmployeeNo,
        ju.FullName,
        ju.IsActive,
        MAX(CASE WHEN ua.AttributeName = 'primary_role' THEN ua.AttributeValue
END) AS primary_role,
        MAX(CASE WHEN ua.AttributeName = 'clearance_level' THEN
ua.AttributeValue END)::int AS clearance_level,
        MAX(CASE WHEN ua.AttributeName = 'approval_limit' THEN
ua.AttributeValue END)::numeric(18,2) AS approval_limit,
        MAX(CASE WHEN ua.AttributeName = 'country_scope' THEN ua.AttributeValue
END) AS country_scope,
        MAX(CASE WHEN ua.AttributeName = 'remote_verifier' THEN
ua.AttributeValue END)::boolean AS remote_verifier
    FROM JobOfferUsers ju
    JOIN UserAttributes ua
        ON ua.JobOfferUserId = ju.Id
    JOIN scenario s
        ON s.employee_no = ju.EmployeeNo
    GROUP BY ju.Id, ju.EmployeeNo, ju.FullName, ju.IsActive

```

```

),
permission_matrix AS (
    SELECT
        p.PermissionCode,
        p.PermissionName,
        up.*,
        s.division_id,
        s.salary,
        s.candidate_country,
        s.is_remote,
        s.remote_verified,
        s.approvals_complete,
        EXISTS (
            SELECT 1
            FROM RolePermissions rp
            WHERE rp.RoleId = up.primary_role
                AND rp.PermissionId = p.Id
        ) AS rbac_granted,
        EXISTS (
            SELECT 1
            FROM UserDivisionAccess uda
            WHERE uda.JobOfferUserId = up.user_id
                AND uda.DivisionId = s.division_id
        ) AS has_division_access
    FROM Permissions p
    CROSS JOIN user_profile up
    CROSS JOIN scenario s
)
SELECT
    PermissionCode,
    rbac_granted,
    CASE
        WHEN IsActive = FALSE THEN TRUE
        WHEN PermissionCode = 'edit_salary' AND primary_role IN ('Recruiter',
'Applicant') THEN TRUE
        WHEN PermissionCode = 'view_salary' AND clearance_level < 3 THEN TRUE
        WHEN PermissionCode = 'approve_offer' AND salary > approval_limit THEN
TRUE
        WHEN PermissionCode = 'approve_offer' AND primary_role = 'DivisionHead'
AND has_division_access = FALSE THEN TRUE
        WHEN PermissionCode = 'publish_offer' AND is_remote = TRUE AND
remote_verified = FALSE THEN TRUE
        WHEN PermissionCode = 'override_policy' AND primary_role <>
'SystemAdmin' THEN TRUE
        ELSE FALSE
    END AS explicit_deny,
    CASE
        WHEN rbac_granted = FALSE THEN FALSE

```

```

    WHEN IsActive = FALSE THEN FALSE
    WHEN PermissionCode = 'create_offer'
        THEN primary_role = 'Recruiter' AND has_division_access = TRUE
    WHEN PermissionCode = 'approve_offer'
        THEN (
            (primary_role = 'HRAdmin'
            OR (primary_role IN ('HiringManager', 'DivisionHead') AND
has_division_access = TRUE))
            AND salary <= approval_limit
            AND (country_scope = 'ALL' OR candidate_country =
ANY(string_to_array(country_scope, '|'))
        )
        WHEN PermissionCode = 'view_salary'
            THEN clearance_level >= 3
        WHEN PermissionCode = 'edit_salary'
            THEN primary_role = 'HRAdmin'
        WHEN PermissionCode = 'verify_remote_work'
            THEN primary_role IN ('Recruiter', 'HRAdmin') AND remote_verifier
= TRUE
        WHEN PermissionCode = 'publish_offer'
            THEN primary_role = 'HRAdmin' AND approvals_complete = TRUE AND
NOT (is_remote = TRUE AND remote_verified = FALSE)
        WHEN PermissionCode = 'override_policy'
            THEN primary_role = 'SystemAdmin'
        ELSE TRUE
    END AS effective_allowed,
    CASE
        WHEN rbac_granted = FALSE THEN 'No RBAC grant for current role'
        WHEN IsActive = FALSE THEN 'Denied because user is inactive'
        WHEN PermissionCode = 'edit_salary' AND primary_role IN ('Recruiter',
'Applicant') THEN 'Explicit deny policy for Recruiter/Applicant salary edits'
        WHEN PermissionCode = 'view_salary' AND clearance_level < 3 THEN
'Explicit deny policy for low salary-clearance'
        WHEN PermissionCode = 'approve_offer' AND salary > approval_limit THEN
'Explicit deny because scenario salary exceeds approval_limit'
        WHEN PermissionCode = 'approve_offer' AND primary_role = 'DivisionHead'
AND has_division_access = FALSE THEN 'Explicit deny because division is outside
user scope'
        WHEN PermissionCode = 'publish_offer' AND is_remote = TRUE AND
remote_verified = FALSE THEN 'Explicit deny until remote verification finishes'
        WHEN PermissionCode = 'override_policy' AND primary_role <>
'SystemAdmin' THEN 'Explicit deny for non-SystemAdmin override attempts'
        WHEN PermissionCode = 'create_offer' AND NOT (primary_role =
'Recruiter' AND has_division_access = TRUE) THEN 'RBAC grant exists, but ABAC
conditions are not satisfied'
        WHEN PermissionCode = 'approve_offer'
            AND NOT (
                (primary_role = 'HRAdmin'

```



```

                OR (primary_role IN ('HiringManager', 'DivisionHead') AND
has_division_access = TRUE))
                AND salary <= approval_limit
                AND (country_scope = 'ALL' OR candidate_country =
ANY(string_to_array(country_scope, '|')))
            ) THEN
'RBAC grant exists, but approval ABAC conditions are not satisfied'
            ELSE 'Allowed'
        END AS decision_reason
FROM permission_matrix
ORDER BY PermissionCode
;

-- =====
-- Validation query 3
-- Show the approval workflow and eligible approvers for a specific offer.
-- Scenario: Engineering offer, USD 165,000, US candidate, remote.
-- DivisionHead becomes required because the salary is above 100,000.
-- Remote verification is an additional publishing prerequisite.
-- =====
WITH offer_input AS (
    SELECT
        20::int AS division_id,
        165000::numeric(18,2) AS salary,
        'US'::text AS candidate_country,
        TRUE::boolean AS is_remote,
        FALSE::boolean AS above_band,
        FALSE::boolean AS remote_verified
),
user_profile AS (
    SELECT
        ju.Id AS user_id,
        ju.EmployeeNo,
        ju.FullName,
        ju.IsActive,
        MAX(CASE WHEN ua.AttributeName = 'primary_role' THEN ua.AttributeValue
END) AS primary_role,
        MAX(CASE WHEN ua.AttributeName = 'approval_limit' THEN
ua.AttributeValue END)::numeric(18,2) AS approval_limit,
        MAX(CASE WHEN ua.AttributeName = 'country_scope' THEN ua.AttributeValue
END) AS country_scope
    FROM JobOfferUsers ju
    JOIN UserAttributes ua
        ON ua.JobOfferUserId = ju.Id
    GROUP BY ju.Id, ju.EmployeeNo, ju.FullName, ju.IsActive
),
eligible_approvers AS (
    SELECT

```

```

        al.SequenceNo,
        al.ApprovalRole,
        up.FullName
FROM ApprovalLevels al
JOIN user_profile up
    ON up.primary_role = al.ApprovalRole
CROSS JOIN offer_input oi
WHERE up.IsActive = TRUE
    AND EXISTS (
        SELECT 1
        FROM RolePermissions rp
        JOIN Permissions p
            ON p.Id = rp.PermissionId
        WHERE rp.RoleId = up.primary_role
            AND p.PermissionCode = 'approve_offer'
    )
    AND (
        up.country_scope = 'ALL'
        OR oi.candidate_country = ANY(string_to_array(up.country_scope,
'|'))
    )
    AND oi.salary <= up.approval_limit
    AND (
        al.ApprovalRole = 'HRAAdmin'
        OR (
            al.ApprovalRole IN ('HiringManager', 'DivisionHead')
            AND EXISTS (
                SELECT 1
                FROM UserDivisionAccess uda
                WHERE uda.JobOfferUserId = up.user_id
                    AND uda.DivisionId = oi.division_id
            )
        )
    )
)
SELECT
    al.SequenceNo,
    al.ApprovalRole,
    CASE
        WHEN al.ApprovalRole = 'DivisionHead'
            AND NOT (oi.salary > 100000 OR oi.above_band = TRUE)
        THEN 'SKIPPED'
        ELSE 'REQUIRED'
    END AS step_status,
    COALESCE(string_agg(DISTINCT ea.FullName, ', ' ORDER BY ea.FullName), '(no
eligible approver)') AS eligible_approvers,
    CASE
        WHEN oi.is_remote = TRUE AND al.ApprovalRole = 'HRAAdmin'

```

```

        THEN 'Remote candidate: verify_remote_work must complete before
publish_offer.'
        ELSE NULL
    END AS notes
FROM ApprovalLevels al
CROSS JOIN offer_input oi
LEFT JOIN eligible_approvers ea
    ON ea.SequenceNo = al.SequenceNo
    AND ea.ApprovalRole = al.ApprovalRole
GROUP BY al.SequenceNo, al.ApprovalRole, oi.salary, oi.above_band, oi.is_remote
ORDER BY al.SequenceNo
;

```

Validation Notes

The first validation query intentionally demonstrates both ordinary and cross-division approval. For the seeded data, a \$150,000 Engineering offer for a US candidate should resolve to Olivia Chen as an HR approver, Sophia Kim as the Engineering DivisionHead approver, and Amelia Torres as a Finance DivisionHead with cross-division Engineering coverage for compensation-committee style review. That gives you one direct division approver and one cross-division approver in the result set, which is useful for testing escalation behavior.

The second query is written as a practical “effective permissions” inspection rather than a generic policy-parser. That is deliberate. The reference material treats policies as stored expressions and the admin mockup emphasizes effective-permission troubleshooting, so the validation query mirrors application-side policy evaluation in SQL-friendly form: start with RBAC grants, apply explicit denies, then apply attribute conditions for the scenario being tested. `filecite{turn0file0}` `filecite{turn0file1}`

The third query turns the sequence in `ApprovalLevels` into a scenario-specific workflow. Because the visible `ApprovalLevels` shape does not expose threshold columns, the query combines routing order with ABAC conditions from the seeded attributes and policies. PostgreSQL’s `WITH` query support is well suited to this kind of layered resolution logic. ³

Inserted Row Counts

The script seeds the following row volumes.

Table	Seeded rows
<code>JobOfferUsers</code>	20
<code>UserAttributes</code>	240
<code>UserDivisionAccess</code>	36
<code>Permissions</code>	31

Table	Seeded rows
RolePermissions	80
AuthorizationPolicies	15
ApprovalLevels	3

Taken together, that is a dense enough sample to exercise the important hybrid cases the reference materials call out: role grants, salary-limited approvals, division scoping, remote verification gates, effective-permission inspection, explicit deny rules, and multi-step routing. [filecite turn0file0](#)
[filecite turn0file1](#)

1 PostgreSQL: Documentation: 18: INSERT

<https://www.postgresql.org/docs/current/sql-insert.html>

2 PostgreSQL: Documentation: 18: 5.5. Constraints

<https://www.postgresql.org/docs/current/ddl-constraints.html>

3 PostgreSQL: Documentation: 18: 7.8. WITH Queries (Common Table Expressions)

<https://www.postgresql.org/docs/current/queries-with.html>